

W4

BLOCK 1 · CLASS 4 · PRACTICAL

Rules, judge, loop.

Last week you watched a feature get built. Today you build one alone — with the three standards every team that ships with agents uses.

What we'll do.

- 1 **Check-in** — your GitHub page: feature 1 merged.
- 2 **What an agent is** — a chat model vs an agent, tool calls, the ReAct loop, the harness, and how we got here.
- 3 **Three standards** — the rulebook, the judge, the loop. Then you use them.
- 4 **Feature 2 kick-off, live** — branch → red → the loop prompt → watch it loop → read the diff.
- 5 **Your turn** — one bug, then feature 2. Thirty-two minutes.
- 6 **Retro** — the rule you add to `AGENTS.md` from what went wrong.

Same groups, but every laptop builds in its own repo today.

Show your GitHub page.

- Repository → branches: `feature-1-balances` or `feature-1-overdue` exists.
- A merged pull request, or the feature commit on `main`.
- Not there yet? Finish it during your turn, before feature 2. The TA notes who.

While the TA walks: pull `main` in GitHub Desktop, run `uv run pytest tests/test_feature_1.py`. Green? You're ready.

A chat model writes. An agent **acts**.

A PLAIN LLM — CHATGPT IN A BROWSER

Text in, text out.

It predicts the next words of an answer. It has never seen your folder — it cannot open a file, run a test, or check anything. *You* copy code in, *you* paste it out, *you* find out whether it works. “The tests pass” is a sentence that sounds right.

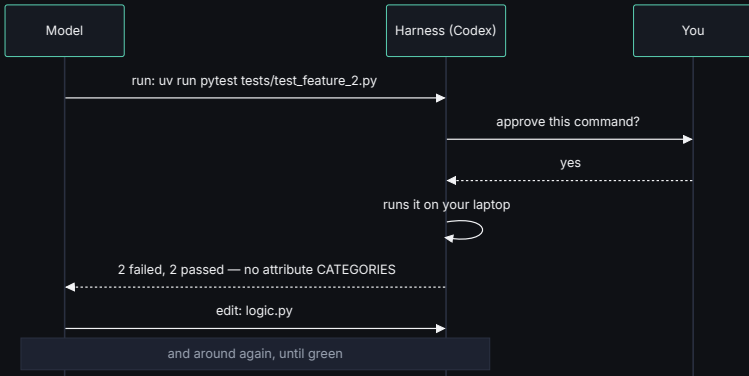
AN AGENT — CODEX ON YOUR REPO

The same model, plus tools, in a loop.

It can **read** your files, **run** commands, **edit** code — and it sees the result of each action before choosing the next. It keeps going until the job is done or it needs you. That is why it asks permission first.

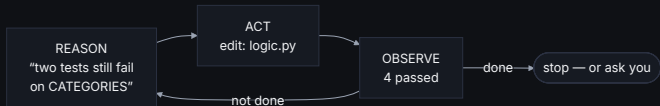
Agent = LLM + tools + a loop. Everything today is about what the tools may do, and how the loop knows when to stop.

How an agent acts: it asks for a tool.



The model writes a **request**, not prose; the harness runs it after the approval prompt and puts the output back. Three tools cover almost everything: **read** a file, **edit** a file, **run** a command.

ReAct: **reason**, **act**, observe. Repeat.



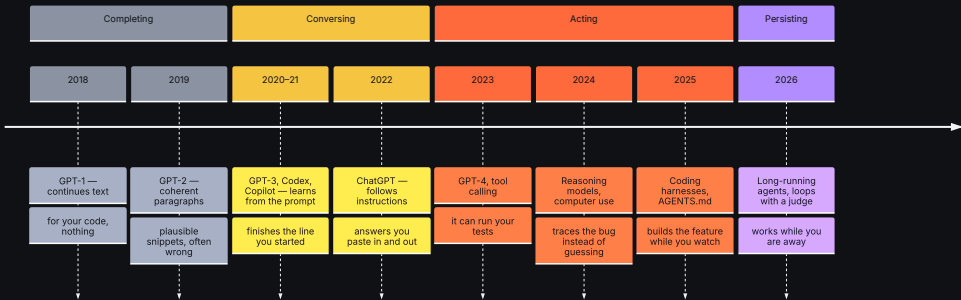
THE HARNESS

The model can only *write* the tool call. Something runs it, shows you the approval prompt, puts the output back, keeps the history, reads `AGENTS.md`, and stops the loop. That software is the **harness**: Codex, Claude Code, Cursor, Copilot's agent.

WHY IT MATTERS

Every agentic coding tool runs this same loop. They differ in the tools they offer and what they ask permission for — not in the loop. Learn the loop once; the tools are interchangeable.

Eight years, four kinds of capability.



The point. The model got better every year. The job changed twice: when it learned to take **instructions**, and when it got **tools and a loop**. Everything after is plumbing and stamina — and a model that can act for hours needs rules and a judge.

AGENTS.md — the file it reads first.

WHAT IT IS

A README for agents

Codex reads it at the start of every session, before anything else. So do twenty other tools. Your repo has had one since Class 2.

WHAT GOES IN

Only what it can't guess

Commands (`uv run ...`).
Boundaries (*never edit tests/*). Etiquette (*branch named after the spec*).
How to report (*paste the test output*).

WHAT STAYS OUT

Anything it can read

Explanations of the code.
Tutorials. "Write clean code." Every extra line hides the important ones.

It is code: in git, reviewed, pruned. When the agent misbehaves, the fix is usually one line here — not a longer prompt.

Three lines. One commit.

Open `AGENTS.md` in your repo. Under **How to answer**, add:

- Build each spec on a branch named after it, e.g. feature-2-categories. Never commit to main directly.
- After every change, run the tests for the task and paste the final output. The output is the proof, not a sentence.
- Never edit, delete or skip anything in tests/. If a test looks wrong, say so and stop.

GitHub Desktop → `AGENTS.md` changed → summary *"AGENTS.md: rules for building features"* → Commit to main → Push.

Define **done** before you build.

SOMETHING THAT CAN'T LIE

The agent stops when the work *looks* done. Give it a check that says pass or fail: the spec's **Done when** and `tests/test_feature_2.py`. Four red tests are your to-do list.

NEVER MOVE THE GOALPOSTS

An agent that edits a test to make it pass has cheated, not finished. The rule is in `AGENTS.md`. You enforce it.

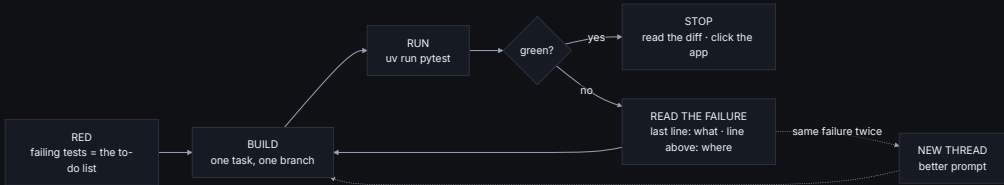
EVIDENCE, NOT CLAIMS

"All tests pass" is a sentence. The test output is evidence. Ask for it every time.

GREEN IS NOT THE END

The judge checks what it was told to check. After green: read the diff, click the app. App wrong, tests green? Write down what the judge missed — that's a test that should exist.

Red → build → run → fix → green.



ONE TASK PER LOOP

One bug, or one spec.
Not “fix everything you see”. The rule is in the file; the prompt names one file.

LET IT LOOP

Tell it to run the tests after every change and keep going until green. It reads the failure and fixes it. You read the result.

A STOP CONDITION

Green — or a cap. Same failure twice: stop, read it yourself, **new thread**, better prompt.

The loop prompt.

Build `specs/feature-2-categories.md`. Work only on the branch `feature-2-categories`. After every change, run `uv run pytest tests/test_feature_2.py` and keep going until all four tests pass. Do not edit anything in `tests/`. When they pass, paste the final test output and list every file you changed.

THE TASK

One spec, by file name.

THE BOUNDARY

One branch. No `tests/`.

THE JUDGE

The test command, every change.

THE EVIDENCE

Output + file list.

Watch it loop. Then don't trust it.

- 1 **Branch.** GitHub Desktop → New branch → `feature-2-categories`.
- 2 **Red.** `uv run pytest tests/test_feature_2.py` → 4 failed. The to-do list.
- 3 **New thread. Paste the loop prompt.** Watch: it edits, runs, reads a failure, edits again. Don't touch anything.
- 4 **Green.** Read its output: four passed? Every file it lists?
- 5 **Read the diff.** Three places must change: `COLUMNS` in `db.py`, every line of `seed/expenses.csv`, then `uv run python seed.py`. Did it run seed? Did it touch `data/` by hand?
- 6 **Click the app.** Add expense → is there a category box? Group page → totals per category? Not tested — only you catch it.
- 7 **Commit, push, PR, merge.** Like last week.

One bug, then feature 2.

SPLITIT

issues/002

→ feature-2-categories

Removing a member crashes the page.
Test `test_bug_002`, then categories on
its branch.

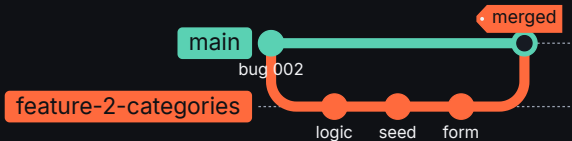
TINY CRM

issues/003

→ feature-2-conversion

"Won this month" is always 0. Test
`test_003`, then conversion by source on
its branch.

Bug on `main`. Feature on its own branch, merged by pull request:



Five things you'll see.

- 1 **It edited a test.** Discard that file in GitHub Desktop. "Never edit `tests/`." Run again.
- 2 **It says done, tests are red.** "Paste the test output." The output is the judge, not the sentence.
- 3 **Same failure three times.** Stop. Read it: last line says what, the line above says where. New thread, better prompt.
- 4 **It fixed three other things too.** Discard them. One task per loop. Ask again for the one thing.
- 5 **Green, but the app is wrong.** The judge missed it. Write down what. Re-prompt with the symptom, or fix by hand.

Every row is a line for your `AGENTS.md`. Keep the list.

What did it get wrong? Write the rule.

- In your group: the worst thing the agent did today. One sentence each.
- Turn it into a rule. Short, specific, testable. *"Run `uv run python seed.py` after changing anything in `seed/`."*
- Add it to `AGENTS.md`. Commit: *"AGENTS.md: <the rule>, because <what happened>".* Push.
- Three groups read theirs out.

That is how the rulebook grows: from what went wrong, one line at a time. Not from a longer prompt.

Block 2 starts: a spec you build alone.

- **Finish feature 2.** Green, merged, pushed.
- **Your rule is in** `AGENTS.md`, committed, with the reason in the message.
- **Push before you close the laptop.** Next week you build on this repo.

Rulebook · judge · loop.

Everything in Block 2 is those three, on your own spec.